

CVE-to-My-Stack Translator

Turn thousands of CVEs into a short, prioritised action list for your stack

CyberHack 2026 · Approach A (Python) + **Web UI**

Your name / team · 5-minute demo

The problem (~30 sec)

- ~23,000 CVEs in the 2026 NVD feed alone — growing daily
- Small IT teams cannot read them all
- Harder still: which ones affect **your** software?

| "47 CVEs overnight — which 4 matter to us?"

Our tool: informal asset list in → ranked table + plain English out

What we deliver

You provide	You get
Informal asset list (tab/file/paste)	Top 50 prioritised CVE rows
e.g. <i>Office 365 Business</i>	CVE, asset, criticality , CVSS, EPSS, KEV
CLI, notebook, or web UI	One-sentence risk summary + CSV download

100% offline — FKIE NVD `CVE-2026.json` , CISA KEV, EPSS (no live APIs)

End-to-end pipeline

```
flowchart LR
    A[Asset list] --> B[Normalise]
    B --> C[Match NVD CPEs]
    C --> D[EPSS + KEV]
    D --> E[Rank + Criticality]
    E --> F[CSV / Web UI]
```

Step	What happens
Normalise	Fuzzy map to vendor:product (hardest)
Match	Filter ~23k 2026 CVEs to your stack (~6 s)
Rank	KEV first → urgency → EPSS → CVSS
Explain	Criticality badge + risk sentence

Offline data (Hour 1–2)

Feed	File	Role
NVD	<code>CVE-2026.json</code> (~22.7k CVEs)	CPE, CVSS, descriptions
CISA KEV	<code>known_exploited_vulnerabilities.json</code>	Confirmed active exploitation
EPSS	<code>epss_scores-YYYY-MM-DD.csv</code>	Exploitation probability
CPE dict	<code>nvdcppe-2.0.zip</code>	Build/verify normalisation map

Download: `python scripts/download_datasets.py`

Normalisation (the hard part)

- `config/normalisation_map.json` — 20 products, fuzzy aliases (`rapidfuzz`)
- Facilitator sample list: 12 / 12 mapped
- Wrong product name → **CVE silently missing** (no error)

Examples:

- `Office 365 Business` → `microsoft:365_apps`
- `Cisco IOS XE 17.9` → `cisco:ios_xe`

MVP: no version-range matching on CPE yet

Ranking & criticality

Sort order (highest row first):

- 1. KEV (yes before no) — patch these first
- 2. Urgency = 40% CVSS + 60% EPSS (+0.25 if KEV)
- 3. EPSS, then CVSS as tie-breakers

Criticality column (plain English for managers):

Label	Rule (simplified)
Critical	CVSS ≥ 9, or KEV + high EPSS/CVSS
High	KEV, or CVSS ≥ 7, or EPSS ≥ 0.5
Medium / Low	Lower scores

Example: CVE-2026-32202 — CVSS 4.3 but KEV + EPSS 0.57 → **Critical**

Web UI (demo option)

FastAPI + Jinja + Tailwind at <http://127.0.0.1:8000>

- Paste or upload asset list · **Load sample** button
- Dataset status cards (NVD / KEV / EPSS)
- Results: stats, **criticality filters**, wrapped table (no sideways scroll)
- **Download CSV** · links to NVD per CVE

```
uvicorn ui.app.main:app --reload --host 127.0.0.1 --port 8000
```

Live demo (~90 sec)

CLI

```
python translate.py data/sample_asset_list.txt --brief
```

Or open the web UI → **Analyse stack**

Expect on sample list

- 12 assets mapped · ~996 raw matches
- Top 50 rows · 13 KEV in the list
- output/prioritised_cves.csv + executive_brief.md

Sample output

CVE	Asset	Crit.	CVSS	EPSS	KEV
CVE-2026-32202	Windows Server 2022	Critical	4.3	0.57	yes
CVE-2026-2441	Google Chrome	High	8.8	0.23	yes

"*CVE-2026-32202 affects Windows Server 2022. EPSS 0.57 indicates **high** exploitation probability. **Actively exploited** (CISA KEV).*"

Who it's for

User	How they use it
SMB sysadmin	Paste stack → patch Critical / KEV first
School / uni IT	Weekly CSV for management reporting
Charity volunteer	Plain English — no CVE jargon required

Quality: 44 pytest tests · CLI · `starter_notebook.ipynb` · web UI

Design choices & honest limits

Why it works

- KEV + EPSS = strong real-world signals, fast in-memory joins
- Vendor pre-filter keeps full NVD scan ~**6 seconds**

Limits (say these aloud)

- Bad alias → **missed CVE**
- Low EPSS $\neq$ safe · Not in KEV $\neq$ unexploited
- **No version-range CPE matching** in MVP

Thank you

CVE-to-My-Stack Translator

Questions?

`translate.py` · `ui/` · `pytest` · [DEMO.md](#) · [REQUIREMENTS_VERIFICATION.md](#)